HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 01

Course Objectives & GenAI/LLM Concepts Recap

Why we're here, what an AI Champion does, and a shared GenAI/LLM vocabulary — climbed from basics to agents — before we baseline how we engineer today.

DURATION
60 minutes

FORMAT
Facilitated discussion — no lab

DELIVERED FOR
Honeywell Software Engineering

How We'll Spend These Sixty Minutes

Six connected moves — from why we're here, through a concept recap, to a measurable baseline.



01

Programme Objectives & Journey

8 min

The business problems this programme solves and how 12 modules build on each other.



02

The AI Champion Role

7 min

What a Champion does beyond using the tools — codify, guard, coach and measure.



03

GenAI & LLM Foundations

12 min

Generative AI, how an LLM predicts the next token, and what that implies for code.



04

Tokens, Context, Prompts & Agents

13 min

The concept ladder up to agents: tokens, context window, prompt anatomy, context engineering, agent loop.



05

Current-State Pain Points

10 min

Where delivery hurts today across the SDLC — mapped against a target-state workflow.



06

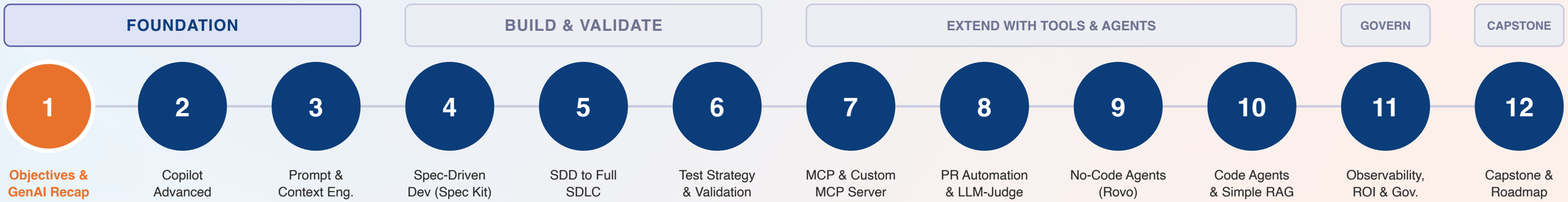
Success Measures & Baseline

10 min

Six baseline metrics we'll prove movement against by the capstone.

Module 01 in the 12-Module Journey

A deliberate engineering progression — each module assumes the one before it.



GREENFIELD	Frame problem →	Specify →	Plan & tasks →	Implement →	Test →	Review & release
BROWNFIELD	Understand repo →	Analyse impact →	Preserve patterns →	Change safely →	Protect regressions →	Modernise

Why this matters: Every later module assumes the shared vocabulary and baseline we set today. The capstone has to prove movement against the measures we agree in this hour.

Six Engineering Problems This Programme Is Built to Solve

The objectives aren't 'learn AI tools' — they're these delivery problems, each owned by specific modules.

Slow implementation cycles

Repetitive requirements, architecture, coding, test, docs, CI/CD and review work stretches delivery.

Fixed in Modules 02–05

Brownfield disruption risk

AI changes that violate existing patterns, break contracts or introduce regressions.

Fixed in Modules 05–06

Prompts instead of specs

Requirements translated loosely into tasks give inconsistent results from prompt-only work.

Fixed in Modules 03–04

Manual testing & PR review

Review bottlenecks; hard to prove spec compliance, integration behaviour and regression safety.

Fixed in Modules 06, 08

Token waste, weak context

Excessive token use and undisciplined context raise AI cost and lower output quality.

Fixed in Modules 03, 11

No visibility into AI value

Little insight into usage, quality, failure patterns, token economics or productivity ROI.

Fixed in Modules 11–12

The AI Champion: More Than a Power User

Champions turn individual AI skill into team capability — and prove it with evidence.

01

Practitioner

Uses Copilot Enterprise at power-user level: agent mode, custom instructions, skills, specs and MCP tools.

Uses it well

02

Codifier

Turns what works into reusable assets — instruction sets, skills, spec packs, prompt templates.

Makes it repeatable

03

Guardian

Holds the line on quality gates, spec compliance, security, governance and human review.

Keeps it safe

04

Multiplier & Measurer

Coaches the team, tracks baseline-to-ROI evidence and owns the adoption roadmap.

Spreads & proves it

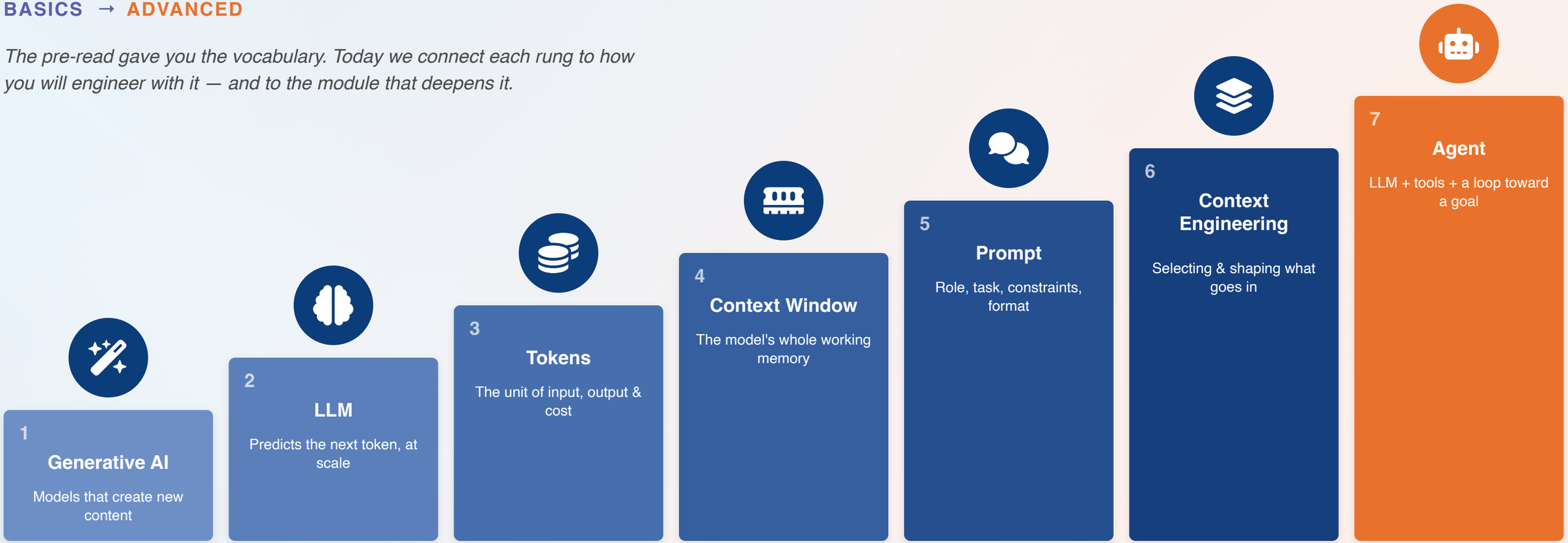
The Champion's job: operationalise judgement, not add caution. You already know how to engineer — the programme makes good AI-assisted practice repeatable, governed and measurable across your team.

The Concept Ladder: From GenAI to Agents

Each rung depends on the one below it — we'll climb it in the next 25 minutes.

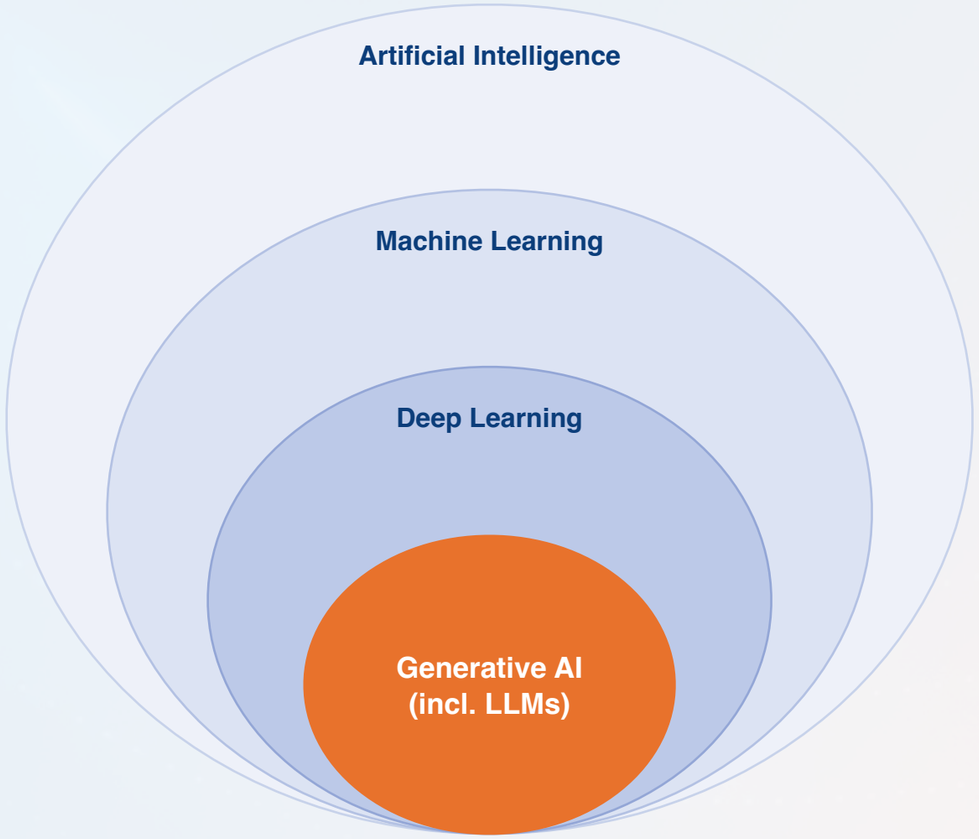
BASICS → ADVANCED

The pre-read gave you the vocabulary. Today we connect each rung to how you will engineer with it — and to the module that deepens it.



Generative AI Creates New Content — It Doesn't Just Classify

Rung 1: where GenAI sits in the AI family, and what changes when a model generates rather than labels.



DISCRIMINATIVE AI

Learns boundaries → picks a label

Sensor reading → Model → Fault | No fault

GENERATIVE AI

Learns patterns → produces new content

Prompt + context → Model → Code | Tests | Specs | Docs

In this programme: GenAI means LLMs generating code, tests, specs and reviews — output that must be verified like any colleague's first draft.

An LLM Is a Next-Token Predictor, Trained at Massive Scale

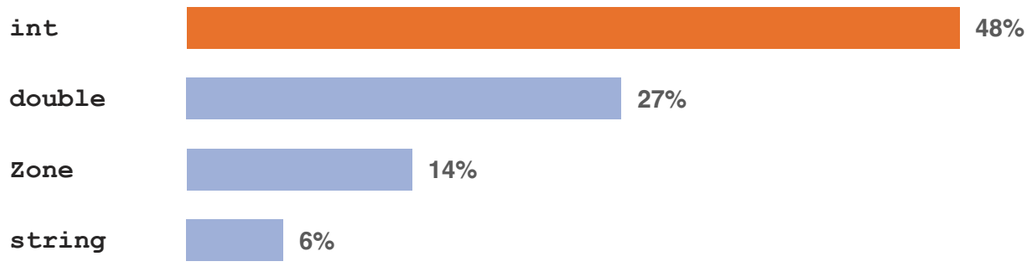
Rung 2: one token at a time, the model picks a likely continuation — then repeats.

THE CODE SO FAR

`public` `void` `Update` `Zone` `Setpoint` `(` `?`

Each box is one token. The model has seen this pattern in vast amounts of code and text.

NEXT-TOKEN PROBABILITIES (illustrative)



HOW AN LLM BECOMES A USEFUL ASSISTANT

Pre-training
Predict the next token across huge volumes of text & code

Instruction tuning
Learn to follow instructions and answer tasks

Preference alignment
Tuned toward helpful, safe, preferred answers

Inference — your turn
Your prompt + context steer what it generates

Implication: fluent is not the same as correct. Output is probable, not verified — which is why specs, tests and review sit at the core of this programme.

Tokens Are the Unit of Everything — Input, Output, Limits, Cost

Rung 3: models don't read words or lines; they read tokens.

TEXT

The	·chiller	·set	point	·drifted	·by	·2	°C	·overnight	.

10 tokens

CODE

def	·get	_zone	_temp	(	self	,	·zone	_id	):

10 tokens

Illustrative split — every model family has its own tokenizer. · marks a leading space, which is part of the token.

≈ 4

chars / token

Rough rule of thumb for English prose. Code, symbols and identifiers usually split finer.

In + Out

both count

You pay for and are limited by input tokens (prompt + context) and output tokens.

Every

limit is tokens

Context window size, rate limits and usage quotas are all measured in tokens.

More ≠

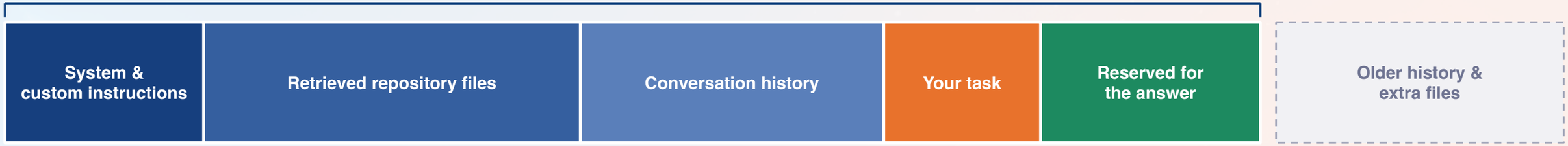
better

Extra tokens raise cost and latency and can dilute focus — context discipline pays twice.

The Context Window Is the Model's Entire Working Memory

Rung 4: a fixed token budget shared by instructions, files, history, your task — and the answer.

CONTEXT WINDOW — fixed limit, counted in tokens



What fills the window is decided by the tool, your instructions and your choices — that's what context engineering controls.

Doesn't fit → dropped, truncated or summarised

Stateless between calls

The model remembers nothing on its own. Everything it knows about your task must be in this window, every time.

Noise dilutes focus

Long, loosely relevant context makes it easier to miss the detail that matters — and costs more tokens.

Temperature: precise ↔ varied



LOW — Code & refactors: keep it low

HIGH — Brainstorming: higher

Anatomy of an Engineering Prompt

Rung 5: six parts that turn a vague request into a task the model can do well.

	EXAMPLE (illustrative)
Role	`You are a senior C#/.NET engineer on a building-automation API.`
Task	`Add input validation to ZoneController.UpdateSetpoint.`
Context	`Validators live in /Validation and use FluentValidation; see ZoneValidator.cs.`
Constraints	`Setpoint range 10–32 °C. No new packages. Don't change the public API contract.`
Examples	`Follow the pattern in AlarmThresholdValidator.cs.`
Output format	`Return a diff plus xUnit tests for valid, boundary and invalid values.`

Rule of thumb: if a new teammate couldn't do the task from your prompt alone, the model can't either. Module 03 recaps prompt patterns: decomposition, iteration and verification.

From Prompt-Only to Context-Engineered

Rung 6: the biggest quality lever isn't a cleverer prompt — it's what you put in the window, and how.

Selected · layered · compressed · task-scoped

Deep-dive: Module 03 | Powers SDD: Module 04

LEVEL 1 — Prompt-only

"Add validation to the setpoint endpoint."

- ! Generic code
- ! Ignores house patterns
- ! Many iterations

→

LEVEL 2 — Prompt + pasted context

Copies whole files and old chat into the prompt.

- ✓ Better fit
- ! Bloated & costly
- ! Inconsistent across people

→

LEVEL 3 — Context-engineered

- **Custom instructions** · *always on*
- **Relevant files & APIs** · *selected*
- **Spec & acceptance criteria** · *task-scoped*
- **Existing tests** · *for verification*

- ✓ Right first time more often
- ✓ Fewer tokens wasted
- ✓ Repeatable across the team

Agents Add a Loop: Plan, Act with Tools, Observe, Adjust

Rung 7: an agent is an LLM that pursues a goal over many steps — using tools, checking results, iterating.

LLM

Answers one question from what's in the window

Assistant

Answers inside your IDE, grounded in your open code

Agent

Takes a goal, plans steps, calls tools, verifies, repeats

Each step adds autonomy — and the need for guardrails.



Repo & files | Terminal & build | Tests | Tools via MCP | Human checkpoint

Agent mode: M02 · MCP: M07 · Build agents: M09–10

Where Delivery Hurts Today — Across the SDLC

Discussion: walk the lifecycle left to right and locate the friction on your team.

Requirements	Design & Architecture	Implementation	Testing	PR Review	Release & Operate
! Tickets and prose translate loosely into implementation tasks; prompt-only results vary by person.	! In brownfield code, AI-suggested changes can bypass existing patterns, contracts and extension points.	! Repetitive coding, boilerplate and docs slow delivery; context is assembled ad hoc each time.	! Test creation is manual and inconsistent; hard to show acceptance criteria are actually covered.	! Review bottlenecks; spec compliance and regression risk are checked by eye, if at all.	! Little visibility into AI usage, quality, failure patterns, token spend or ROI.

Ask the room: Which stage costs your team the most time per sprint? Where has AI already helped — and where has it created rework? Capture answers on the workflow canvas.

Workshop: Map Today's Workflow Against the Target

Facilitated discussion on the current/target-state workflow canvas — no lab, no code.

1 Map today — 4 min

2 Sketch the target — 4 min

3 Pick one measure to baseline — 2 min

TODAY — what typically happens	TARGET — AI-assisted, spec-led
Requirements arrive as tickets and prose	Spec with acceptance criteria drives plan & tasks
Each engineer prompts ad hoc, from scratch	Shared instructions, skills and engineered context
Tests written manually after the code	Test strategy derived from the spec; mocks & stubs
PRs wait in a review queue; checked by eye	AI review + LLM-as-Judge, then human approval
AI usage and token cost are invisible	Traced usage; cost per accepted output tracked

Rows are starting prompts — replace them with what your team actually does.

Six Baseline Measures We'll Prove Movement Against

Capture today's number now — the capstone and ROI model compare against it.

Implementation cycle time

Measure Ticket-ready → merged, for a representative feature

Source Issue tracker & GitHub timestamps

Baseline today:

Quality & regression risk

Measure Defects and regressions escaping to later stages

Source Bug tracker, CI failure history

Baseline today:

PR review time

Measure PR opened → approved; review rounds per PR

Source GitHub pull-request data

Baseline today:

Testing effort

Measure Hours to write/update tests per feature; criteria covered

Source Team estimate + coverage report

Baseline today:

Token usage

Measure Tokens per task and per accepted change

Source Copilot usage data; Agent Prism (M11)

Baseline today:

AI cost

Measure Cost per accepted, production-quality output

Source Usage & billing data; ROI model (M11)

Baseline today:

Module 01 Outcomes & What's Next

What should be true as we leave this hour.

- ✓ Programme objectives, the 12-module journey and the AI Champion role are clear
- ✓ GenAI, LLMs and next-token prediction recapped as a shared vocabulary
- ✓ Tokens, the context window and temperature understood as engineering constraints
- ✓ Prompt anatomy, context engineering and the agent loop linked to the modules that deepen them
- ✓ Current-state pain points mapped against a target-state workflow
- ✓ Six baseline measures recorded: cycle time, quality, PR review, testing, tokens, cost

NEXT: MODULE 02 — GITHUB COPILOT RECAP & ADVANCED FEATURES · 2.5 HOURS

A quick recap of core Copilot, then hands-on power-user work: custom instructions, custom agents, reusable skills, advanced prompting and agent-mode workflows.

THE LADDER WE CLIMBED

GenAI → LLM → Tokens → Context window → Prompt → Context eng. → **Agent**

Questions & Discussion

Module 01 — Course Objectives & GenAI/LLM Concepts Recap